

ReqRes API - QA Assignment

API test cases, negative testing and exploratory contract validation

1. Test Setup

Base URL: https://reqres.in

Tool: Postman

Authentication header:

x-api-key: <API_KEY>

Common headers:

Accept: application/json

Content-Type: application/json (for POST/PUT requests)

Note: The real API key is intentionally not included in the submitted files. Set the apiKey variable in the Postman environment before running the collection.

2. Assignment 1 - Test Cases for a Real Endpoint

The following test cases cover five happy-path scenarios and four negative/non-2xx scenarios. Each expected response contract lists the key fields and their data types, as required by the assignment.

ID	Scenario	Expected status	Type
TC1	Get paginated list of users	200 OK	Happy path
TC2	Get existing user by ID	200 OK	Happy path
TC3	Create a new user	201 Created	Happy path
TC4	Update an existing user	200 OK	Happy path
TC5	Delete an existing user	204 No Content	Happy path
TC6	Get non-existing user	404 Not Found	Negative
TC7	Get non-existing resource	404 Not Found	Negative
TC8	Login without password	400 Bad Request	Negative
TC9	Register without email	400 Bad Request	Negative

2.1 Happy Path Test Cases

TC1 - Get a paginated list of users

Method + URL: GET https://reqres.in/api/users?page=2

Headers:

x-api-key: <API_KEY>

Accept: application/json

Request Body:

None

Expected HTTP Status: 200 OK

Expected Response Contract:

```
page: number (must equal 2)
per_page: number
total: number
total_pages: number
data: array<object>
data[].id: number
data[].email: string
data[].first_name: string
data[].last_name: string
data[].avatar: string
```

TC2 - Get a single user by valid ID

Method + URL: GET https://reqres.in/api/users/2

Headers:

```
x-api-key: <API_KEY>
Accept: application/json
```

Request Body:

None

Expected HTTP Status: 200 OK

Expected Response Contract:

```
data: object
data.id: number (must equal 2)
data.email: string
data.first_name: string
data.last_name: string
data.avatar: string
```

TC3 - Create a new user

Method + URL: POST https://reqres.in/api/users

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "name": "Jan Kowalski",
  "job": "QA Engineer"
}
```

Expected HTTP Status: 201 Created

Expected Response Contract:

```
name: string (echoes the submitted value)
job: string (echoes the submitted value)
id: string (generated, non-empty)
createdAt: string (valid ISO 8601 date-time)
```

TC4 - Fully update an existing user

Method + URL: PUT https://reqres.in/api/users/2

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "name": "Jan Kowalski",
  "job": "Senior QA Engineer"
}
```

Expected HTTP Status: 200 OK

Expected Response Contract:

```
name: string (echoes the submitted value)
job: string (echoes the submitted value)
updatedAt: string (valid ISO 8601 date-time)
```

TC5 - Delete an existing user

Method + URL: DELETE https://reqres.in/api/users/2

Headers:

```
x-api-key: <API_KEY>
Accept: application/json
```

Request Body:

None

Expected HTTP Status: 204 No Content

Expected Response Contract:

No response body. The response text must be an empty string.

2.2 Negative / Non-2xx Test Cases

TC6 - Get a non-existing user

Method + URL: GET https://reqres.in/api/users/99999

Headers:

```
x-api-key: <API_KEY>
Accept: application/json
```

Request Body:

None

Expected HTTP Status: 404 Not Found

Expected Response Contract:

Response body: object with zero properties (exactly {}).

No data field must be returned.

TC7 - Get a non-existing resource

Method + URL: GET https://reqres.in/api/unknown/99999

Headers:

```
x-api-key: <API_KEY>
Accept: application/json
```

Request Body:

None

Expected HTTP Status: 404 Not Found

Expected Response Contract:

Response body: object with zero properties (exactly {}).

No data field must be returned.

TC8 - Login without password

Method + URL: POST https://reqres.in/api/login

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "email": "george.bluth@reqres.in"
}
```

Expected HTTP Status: 400 Bad Request

Expected Response Contract:

error: string (non-empty)

token: must be absent

TC9 - Register without email

Method + URL: POST https://reqres.in/api/register

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "password": "Test123!"
}
```

Expected HTTP Status: 400 Bad Request

Expected Response Contract:

error: string (non-empty)

token: must be absent

3. Assignment 2 - Contract Validation & Exploratory Testing

The requests below were executed in Postman. The status codes and response bodies are the actual observed results.

ID	Scenario	Actual status	Observed behaviour
R1	Wrong data type	200 OK	Password number accepted; token returned
R2	Missing required field	400 Bad Request	Missing password rejected
R3	Invalid resource ID	404 Not Found	Missing resource; empty JSON body
R4	Extra / unexpected fields	201 Created	Additional fields accepted and echoed
R5	Empty values	400 Bad Request	Empty credentials rejected

R1 - Wrong data type

Scenario: Send the password field using the wrong data type (number instead of string).

Method + URL: POST <https://reqres.in/api/login>

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "email": "george.bluth@reqres.in",
  "password": 123456
}
```

Actual HTTP Status: 200 OK

Actual Response Body:

```
{
  "token": "QpwL5tke4Pnpja7X1",
  "_meta": {
    "powered_by": "ReqRes",
    "docs_url": "https://app.reqres.in/documentation",
    "upgrade_url": "https://app.reqres.in/upgrade",
    "example_url": "https://app.reqres.in/examples/notes-app",
    "variant": "v1_b",
    "message": "This is a read-only demo endpoint. Sign up to create your own collections with full CRUD and auth.",
    "cta": {
      "label": "Get started",
      "url": "https://app.reqres.in/upgrade"
    },
    "context": "legacy_success"
  }
}
```

Analysis: The API accepted a numeric password and returned a token, showing that the endpoint does not strictly validate this field data type. A stricter contract would reject the request with a 4xx validation error.

R2 - Missing required field

Scenario: Omit the required password field.

Method + URL: POST <https://reqres.in/api/login>

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "email": "george.bluth@reqres.in"
}
```

Actual HTTP Status: 400 Bad Request

Actual Response Body:

```
{
  "error": "Missing password"
}
```

Analysis: The API correctly rejected the request because the required password field was missing. The 400 Bad Request status is appropriate, and the error message clearly identifies the missing field.

R3 - Invalid resource ID

Scenario: Target a user resource that does not exist.

Method + URL: GET https://reqres.in/api/users/99999

Headers:

```
x-api-key: <API_KEY>
Accept: application/json
```

Request Body:

None

Actual HTTP Status: 404 Not Found

Actual Response Body:

```
{}
```

Analysis: The API returned 404 Not Found for a user ID that does not exist, which is the appropriate HTTP status for a missing resource. However, the response body is an empty JSON object. A more descriptive error response could make the API easier for developers to debug and integrate with.

R4 - Extra / unexpected fields

Scenario: Add fields that are not part of the basic user creation data.

Method + URL: POST https://reqres.in/api/users

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "name": "Jan Kowalski",
  "job": "QA Engineer",
  "isAdmin": true,
  "unexpectedField": "test"
}
```

Actual HTTP Status: 201 Created

Actual Response Body:

```
{
  "name": "Jan Kowalski",
  "job": "QA Engineer",
  "isAdmin": true,
  "unexpectedField": "test",
  "id": "810",
  "createdAt": "2026-08-07T08:57:22.615Z",
  "_meta": {
    "powered_by": "ReqRes",
    "docs_url": "https://app.reqres.in/documentation",
    "upgrade_url": "https://app.reqres.in/upgrade",
    "example_url": "https://app.reqres.in/examples/notes-app",
    "variant": "v1_b",
    "message": "This is a read-only demo endpoint. Sign up to create your own collections with full CRUD and auth.",
    "cta": {
      "label": "Get started",
      "url": "https://app.reqres.in/upgrade"
    },
    "context": "legacy_success"
  }
}
```

Analysis: The API accepted the request with additional fields that were not part of the basic user creation data. It also returned those unexpected fields in the response, which shows permissive request validation. This can be acceptable if additional properties are intentionally supported, but the API contract should document this behaviour clearly.

R5 - Empty required values

Scenario: Send required login fields as empty strings.

Method + URL: POST https://reqres.in/api/login

Headers:

```
x-api-key: <API_KEY>
Content-Type: application/json
Accept: application/json
```

Request Body:

```
{
  "email": "",
  "password": ""
}
```

Actual HTTP Status: 400 Bad Request

Actual Response Body:

```
{
  "error": "Missing email or username"
}
```

Analysis: The API correctly rejected the empty credentials with 400 Bad Request. The error message identifies the email or username issue, but it does not mention that the password is also empty.

4. Conclusions

Assignment 1: All nine Postman test cases passed. The final collection adds explicit type assertions, validates every returned user, checks ISO 8601 timestamps, and requires exact empty objects for TC6 and TC7.

Assignment 2: The exploratory tests confirmed appropriate 4xx handling for missing fields, empty credentials and invalid resource IDs. They also exposed permissive validation: numeric passwords and unexpected user fields were accepted.